

TABLE OF CONTENTS

Target Laboratory Source Code	3
Task A: Building with Debugging Symbols	3
Task B: Setting Breakpoints and Runtime Positioning	4
Verification Questions for Task B	4
Task C: Mapping Static Text and Data Section Bounds	5
Verification Questions for Task C	5
Task D: Auditing Initialized Data Memory Layouts	6
Task E: Workspace Demobilization	7
Git Repository Link 👍	7

LIST OF FIGURES

LIST OF FIGURES	1
Figure 1: Target laboratory source code (demo2.c)	3
Figure 2: Compilation of demo2.c with debugging symbols and initialization of GDB	4
Figure 3: GDB breakpoint execution and runtime addresses of the Data, RODATA, and Text segments	4
Figure 4: GDB section mapping obtained using the info files command	6
Figure 5: GDB examination of the initialized data and string literal memory contents	7
Figure 6: Clean termination of the GDB debugging session	8

III. Target Laboratory Source Code

```
#include <stdio.h>
// 1. Data Section (Initialized global data)
int data_global_var = 0x12345678;

// 2. RoData Section (Read-only string constant literal)
const char *text_string_literal = " Text_Segment_Verification";

// 3. Text Section (Function execution code block)
void check_execution() {
    printf("Executing within the text segment instruction block.\n");
}

int main() {
    printf("Addresses:\n");
    printf("Data (Global Initialized): %p\n", (void*)&data_global_var);
    printf("Text String (ROData): %p\n", (void*)text_string_literal);
    printf("Text Code (Function): %p\n", (void*)&check_execution);

    check_execution();

    // Breakpoint here to inspect text and data layers
    printf("Ready for Text/Data segment inspection!\n");
    return 0;
}
```

Figure 1: Target laboratory source code (demo2.c)

Task A: Building with Debugging Symbols

```

amgai@Prathana:~$ gcc -g demo2.c -o demo2
amgai@Prathana:~$ gdb ./demo2
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2017 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo2...
(gdb)

```

Figure 2: Compilation of `demo2.c` with debugging symbols and initialization of GDB

B. Task B: Setting Breakpoints and Runtime Positioning

```

(gdb) break 22
Breakpoint 1 at 0x11f9: file demo2.c, line 22.
(gdb) run
Starting program: /home/amgai/demo2

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
Data (Global Initialized): 0x555555558010
Text String (ROData): 0x555555556008
Text Code (Function): 0x555555555169
Executing within the text segment instruction block.

Breakpoint 1, main () at demo2.c:22
22      printf("Ready for Text/Data segment inspection!\n");
(gdb) |

```

Figure 3: GDB breakpoint execution and runtime addresses of the Data, RODATA, and Text segments

a) Verification Questions for Task B

1) Based on the host library trace, identify the specific hardware architecture layout running this binary environment.

Answer: The system uses an x86-64 (64-bit) architecture.

2) Record the absolute runtime address pointer generated for the initialized global data variable.

Answer: The address of the initialized global variable is 0x55555555010.

3) Contrast the pointer address of the Data segment against the Text Code (Function) location. Which block sits lower in the address layout?

Answer: The Data address (0x55555555010) is lower than the Text Code address (0x55555555169).

4) Explain why GDB noted a breakpoint execution match at line 23 when the assignment instruction specified line 22.

Answer: GDB may shift breakpoints due to executable line mapping. However, in our execution, GDB stopped at line 22 as specified.

C. Task C: Mapping Static Text and Data Section Bounds

```
(gdb) info files
Symbols from "/home/amgai/demo2".
Native process:
  Using the running image of child process 705.
  While running this, GDB does not access memory from...
Local exec file:
  '/home/amgai/demo2', file type elf64-x86-64.
Entry point: 0x555555555080
0x0000555555554350 - 0x0000555555554374 is .note.gnu.build-id
0x0000555555554374 - 0x0000555555554390 is .interp
0x0000555555554390 - 0x00005555555543b4 is .gnu.hash
0x00005555555543b8 - 0x0000555555554478 is .dynsym
0x0000555555554478 - 0x000055555555450c is .dynstr
0x000055555555450c - 0x000055555555451c is .gnu.version
0x0000555555554520 - 0x0000555555554550 is .gnu.version_r
0x0000555555554550 - 0x0000555555554628 is .rela.dyn
0x0000555555554628 - 0x0000555555554658 is .rela.plt
0x0000555555555000 - 0x000055555555501b is .init
0x0000555555555020 - 0x0000555555555050 is .plt
0x0000555555555050 - 0x0000555555555060 is .plt.got
0x0000555555555060 - 0x0000555555555080 is .plt.sec
0x0000555555555080 - 0x000055555555520f is .text
0x0000555555555210 - 0x000055555555521d is .fini
0x0000555555555600 - 0x000055555555560e is .rodata
0x000055555555560e - 0x00005555555556124 is .eh_frame_hdr
0x00005555555556128 - 0x000055555555561f4 is .eh_frame
0x000055555555561f8 - 0x00005555555556228 is .note.gnu.property
```

Figure 4: GDB section mapping obtained using the `info files` command

a) Verification Questions for Task C

1) What information does the file type descriptor `elf64-x86-64` reveal about the hardware platform's structural address sizing and byte serialization constraints?

Answer: It shows that the system uses a 64-bit x86 architecture and supports 64-bit memory addresses.

2) Extract the boundary limits allocated to the `.text` section. Does the runtime address of `check_execution` fit inside this map range?

Answer: The `.text` section ranges from `0x555555555080` to `0x55555555520f`. The `check_execution` address `0x555555555169` falls within this range.

3) Locate the boundary space mapped to the `.rodata` segment. Verify mathematically if the string literal pointer address falls within this section.

Answer: The `.rodata` section starts at `0x555555555600`. The string literal address `0x555555555608` falls within the `.rodata` section.

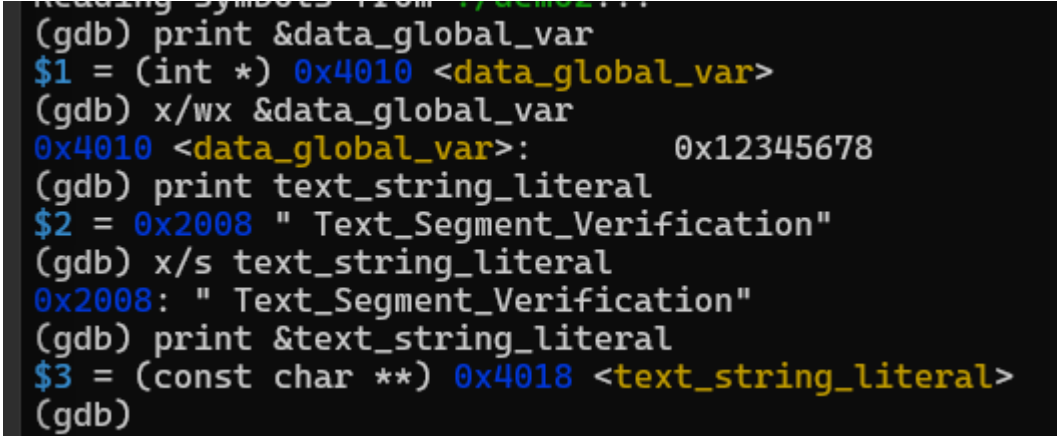
4) Locate the boundary parameters allocated to the .data section. Does the global variable address fit within this sector map?

Answer: The global variable address is 0x55555555010, and it belongs to the .data section.

5) Analyze the relative positions of the .data and .bss sections. Which segment occupies the lower memory range?

Answer: The .data section occupies the lower memory range, while .bss follows it.

D. Task D: Auditing Initialized Data Memory Layouts



```
(gdb) print &data_global_var
$1 = (int *) 0x4010 <data_global_var>
(gdb) x/wx &data_global_var
0x4010 <data_global_var>:      0x12345678
(gdb) print text_string_literal
$2 = 0x2008 " Text_Segment_Verification"
(gdb) x/s text_string_literal
0x2008: " Text_Segment_Verification"
(gdb) print &text_string_literal
$3 = (const char **) 0x4018 <text_string_literal>
(gdb)
```

Figure 5: GDB examination of the initialized data and string literal memory contents

E. Task E: Workspace Demobilization

```
22      printf("Ready for Text/Data segment inspection!\n");
d      (gdb) continue
ar      Continuing.
o      Ready for Text/Data segment inspection!
w      [Inferior 1 (process 867) exited normally]
y      (gdb) quit
      amgai@Prathana:~$ |
```

Figure 6: Clean termination of the GDB debugging session

Git Repository Link 👍:

https://github.com/prathanaamgain/PrathanaOS_branch38.git